Event-Driven Architecture (Asynchronous Systems)

Theory, Edge Cases & Code Examples

- ✓ What is Event-Driven Architecture? (Simple Explanation)
- ✓ Core Components of Event-Driven Systems
- ✓ Event-Driven Patterns
- ✓ Synchronous vs Asynchronous Event Handling
- ✓ Event-Driven in Real-World C++ Systems

→ *Swipe to learn* →

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 1. What is Event-Driven Architecture? (Simple Explanation) (1/3)

Think of it like a post office:

Traditional (Synchronous)	Event-Driven (Asynchronous)
You walk to friend's house	You mail a letter
Wait at door for answer	Don't wait, continue your day
Blocking - can't do other things	Non-blocking - do multiple things
Direct coupling - know friend's address	Loose coupling - post office handles delivery

Event-Driven Architecture (EDA) = Systems that react to events

Key Concepts:

Concept	What It Means	Example
Event	Something happened	"Button clicked", "Sensor reading arrived", "File uploaded"
Producer	Creates events	Button, sensor, user action
Consumer	Reacts to events	Display update, data processing, notification
Event Bus	Delivers events	Central communication channel
Asynchronous	Don't wait for response	Publish event and continue

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 1. What is Event-Driven Architecture? (Simple Explanation) (2/3)

Simple Example:

```
// Traditional (synchronous, blocking)
void onClick() {
    updateDatabase();    // Wait for DB
    sendEmail();         // Wait for email
    updateUI();          // Wait for UI
    // Total time: DB + Email + UI (sequential)
}

// Event-Driven (asynchronous, non-blocking)
void onClick() {
    eventBus.publish(ClickEvent{});  // Fire and forget
    // Returns immediately!
    // Handlers process events independently
}

// Handlers run independently
void onClickEvent() {
    updateDatabase();  // Runs in parallel
}
void onClickEvent2() {
    sendEmail();       // Runs in parallel
}
void onClickEvent3() {
    updateUI();        // Runs in parallel
}
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 1. What is Event-Driven Architecture? (Simple Explanation) (3/3)

When to Use Event-Driven:

✅ Use Event-Driven When:

- GUI applications (user interactions)
- Real-time systems (sensors, trading)
- Microservices (decoupled communication)
- IoT systems (many devices publishing data)
- Notification systems (many subscribers)

❌ Don't Use Event-Driven When:

- Simple sequential logic (calculator)
- Need immediate response value
- Simple request-response (function calls fine)
- Debugging/tracing critical (events harder to trace)

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 2. Core Components of Event-Driven Systems (1/3)

The 4 Building Blocks:

```
┌──────────────┐
│  Producer    │ ─── Creates Event ──▶ ┌──────────────┐
│ (Publishes)  │                       │  Event Bus   │
└──────────────┘                       │  (Routes)    │
                                       └──────┬───────┘
                                              │
                  ┌───────────────────────────┼───────────────────────────┐
                  ▼                           ▼                           ▼
            ┌──────────────┐            ┌──────────────┐            ┌──────────────┐
            │ Consumer1    │            │ Consumer2    │            │ Consumer3    │
            │(Subscribes)  │            │(Subscribes)  │            │(Subscribes)  │
            └──────────────┘            └──────────────┘            └──────────────┘
```

1. Event (The Message)

```
struct Event {
    EventType type;        // What kind of event?
    std::string source;    // Who created it?
    void* data;            // Payload (actual data)
    uint64_t timestamp;    // When did it happen?
};

// Example events
Event buttonClick{EventType::ButtonClick, "LoginButton", nullptr, now()};
Event sensorData{EventType::SensorReading, "TempSensor", &tempData, now()};
Event fileUploaded{EventType::FileUploaded, "UploadService", &fileInfo, now()};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 2. Core Components of Event-Driven Systems (2/3)

2. Producer (Event Creator)

```
class Button {
    EventBus* bus;

public:
    void onClick() {
        std::cout << "Button clicked!\n";

        // Publish event (fire and forget)
        Event e{EventType::ButtonClick, "MyButton", nullptr, now()};
        bus->publish(e);

        // Returns immediately, doesn't wait for handlers
    }
};
```

3. Event Bus (Message Router)

The central hub that:
- Receives events from producers
- Routes to interested consumers
- Manages subscriptions
- Handles queuing (optional)

```
class EventBus {
    std::map<EventType, std::vector<EventHandler>> subscribers;

public:
    // Subscribe to events
    void subscribe(EventType type, EventHandler handler) {
        subscribers[type].push_back(handler);
    }

    // Publish event to all subscribers
    void publish(const Event& event) {
        auto it = subscribers.find(event.type);
        if (it != subscribers.end()) {
// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 2. Core Components of Event-Driven Systems (3/3)

[Code continues across slides]

```
// ...continued from previous slide
            for (auto& handler : it->second) {
                handler(event);  // Call each subscriber
            }
        }
    }
};
```

4. Consumer (Event Handler)

```
class Logger {
public:
    Logger(EventBus* bus) {
        // Subscribe to button clicks
        bus->subscribe(EventType::ButtonClick, [](const Event& e) {
            std::cout << "[LOG] Button clicked at " << e.timestamp << "\n";
        });
    }
};

class Analytics {
public:
    Analytics(EventBus* bus) {
        // Subscribe to same events
        bus->subscribe(EventType::ButtonClick, [this](const Event& e) {
            trackClick(e);
        });
    }

    void trackClick(const Event& e) {
        clickCount++;
        std::cout << "[ANALYTICS] Total clicks: " << clickCount << "\n";
    }

private:
    int clickCount = 0;
};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 3. Event-Driven Patterns (1/4)

Pattern 1: Simple Event Bus (In-Memory)

```
// Synchronous event delivery (handlers called immediately)

class SimpleEventBus {
    std::map<EventType, std::vector<std::function<void(const Event&)>>> handlers;

public:
    void subscribe(EventType type, std::function<void(const Event&)> handler) {
        handlers[type].push_back(handler);
    }

    void publish(const Event& event) {
        // Call all handlers synchronously
        for (auto& handler : handlers[event.type]) {
            handler(event);  // Blocks until handler completes
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 3. Event-Driven Patterns (2/4)

Pattern 2: Queued Event Bus (Async)

```
// Asynchronous event delivery (handlers called later)

class QueuedEventBus {
    std::queue<Event> eventQueue;
    std::mutex queueMutex;
    std::map<EventType, std::vector<EventHandler>> handlers;

public:
    void subscribe(EventType type, EventHandler handler) {
        handlers[type].push_back(handler);
    }

    void publish(const Event& event) {
        // Add to queue and return immediately
        std::lock_guard lock(queueMutex);
        eventQueue.push(event);
    }

    void processEvents() {
        std::lock_guard lock(queueMutex);

        while (!eventQueue.empty()) {
            Event e = eventQueue.front();
            eventQueue.pop();

            // Process event (call handlers)
            for (auto& handler : handlers[e.type]) {
                handler(e);
            }
        }
    }
};

// Usage: Call processEvents() periodically (e.g., in game loop)
while (running) {
    // Handle input
    // Update game logic
    eventBus.processEvents();  // Process accumulated events
    // Render
}
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 3. Event-Driven Patterns (3/4)

Pattern 3: Thread-Based Event Bus (Parallel)

```
// Each event processed in separate thread

class ThreadedEventBus {
    std::map<EventType, std::vector<EventHandler>> handlers;
    std::mutex handlersMutex;

public:
    void subscribe(EventType type, EventHandler handler) {
        std::lock_guard lock(handlersMutex);
        handlers[type].push_back(handler);
    }

    void publish(const Event& event) {
        // Create copy of handlers (avoid holding lock during callbacks)
        std::vector<EventHandler> handlersCopy;
        {
            std::lock_guard lock(handlersMutex);
            auto it = handlers.find(event.type);
            if (it != handlers.end()) {
                handlersCopy = it->second;
            }
        }

        // Launch thread for each handler
        for (auto& handler : handlersCopy) {
            std::thread([handler, event]() {
                handler(event);
            }).detach();
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 3. Event-Driven Patterns (4/4)

Pattern 4: Priority Event Bus (Ordered)

```
// Events with priority (higher priority processed first)

struct PriorityEvent {
    Event event;
    int priority;  // Higher = more urgent

    bool operator<(const PriorityEvent& other) const {
        return priority < other.priority;  // Max heap
    }
};

class PriorityEventBus {
    std::priority_queue<PriorityEvent> eventQueue;
    std::mutex queueMutex;

public:
    void publish(const Event& event, int priority) {
        std::lock_guard lock(queueMutex);
        eventQueue.push({event, priority});
    }

    void processEvents() {
        std::lock_guard lock(queueMutex);

        while (!eventQueue.empty()) {
            PriorityEvent pe = eventQueue.top();
            eventQueue.pop();

            // Process highest priority event first
            processEvent(pe.event);
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 4. Synchronous vs Asynchronous Event Handling (1/2)

Synchronous (Immediate, Blocking):

```
void publish(const Event& event) {
    for (auto& handler : handlers[event.type]) {
        handler(event);  // ← Blocks here until handler completes
    }
    // All handlers done before returning
}

// Characteristics:
✅ Simple to understand (linear flow)
✅ Handlers execute in order
✅ Easy to debug
❌ Slow if handlers take time
❌ One slow handler blocks all others
```

Asynchronous (Deferred, Non-Blocking):

```
void publish(const Event& event) {
    eventQueue.push(event);  // ← Returns immediately
    // Handlers called later
}

void processEvents() {
    while (!eventQueue.empty()) {
        Event e = eventQueue.front();
        eventQueue.pop();
        // Now call handlers
        for (auto& handler : handlers[e.type]) {
            handler(e);
        }
    }
}

// Characteristics:
✅ Fast publish (non-blocking)
✅ Handlers can't slow down producer
✅ Can batch process events
❌ More complex (when are handlers called?)
❌ Harder to debug (delayed execution)
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 4. Synchronous vs Asynchronous Event Handling (2/2)

Comparison Table:

Aspect	Synchronous	Asynchronous
Performance	Slower (waits for handlers)	Faster (returns immediately)
Order	Handlers run in order	Handlers may run out of order
Debugging	Easy (call stack)	Harder (delayed execution)
Error Handling	Immediate (can catch)	Delayed (need error queue)
Use Case	Simple apps, single thread	Real-time, multi-threaded

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 5. Event-Driven in Real-World C++ Systems (1/5)

Example 1: GUI Applications (Qt-style Signals/Slots)

```
class Button {
    std::vector<std::function<void()>> clickHandlers;

public:
    // Connect handler (subscribe)
    void connect(std::function<void()> handler) {
        clickHandlers.push_back(handler);
    }

    // Emit signal (publish)
    void click() {
        for (auto& handler : clickHandlers) {
            handler();  // Call all connected slots
        }
    }
};

// Usage
Button loginButton;

// Connect multiple handlers
loginButton.connect([]() { std::cout << "Logging in...\n"; });
loginButton.connect([]() { std::cout << "Analytics: Login clicked\n"; });
loginButton.connect([]() { std::cout << "Sound: Play click sound\n"; });

loginButton.click();
// Output:
// Logging in...
// Analytics: Login clicked
// Sound: Play click sound
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 5. Event-Driven in Real-World C++ Systems (2/5)

Example 2: Game Engine (Entity Events)

```
enum class GameEvent {
    PlayerDied,
    EnemySpawned,
    LevelCompleted,
    ItemCollected
};

class GameEventBus {
    std::map<GameEvent, std::vector<std::function<void()>>> handlers;

public:
    void on(GameEvent event, std::function<void()> handler) {
        handlers[event].push_back(handler);
    }

    void trigger(GameEvent event) {
        for (auto& handler : handlers[event]) {
            handler();
        }
    }
};

// Usage in game
GameEventBus bus;

// Different systems subscribe
bus.on(GameEvent::PlayerDied, []() {
    std::cout << "Audio: Play death sound\n";
});

bus.on(GameEvent::PlayerDied, []() {
    std::cout << "UI: Show game over screen\n";
});

bus.on(GameEvent::PlayerDied, []() {
    std::cout << "Analytics: Track player death\n";
});

// Game triggers event
// ...continues on next slide
```

📌 5. Event-Driven in Real-World C++ Systems (3/5)

[Code continues across slides]

```
// ...continued from previous slide
if (player.health <= 0) {
    bus.trigger(GameEvent::PlayerDied);
}
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 5. Event-Driven in Real-World C++ Systems (4/5)

Example 3: IoT System (Sensor Events)

```
struct SensorReading {
    std::string sensorId;
    double value;
    uint64_t timestamp;
};

class IoTEventBus {
    std::map<std::string, std::vector<std::function<void(const SensorReading&)>>> subscribers;

public:
    void subscribe(const std::string& sensorId,
                   std::function<void(const SensorReading&)> handler) {
        subscribers[sensorId].push_back(handler);
    }

    void publish(const SensorReading& reading) {
        // Notify all subscribers for this sensor
        auto it = subscribers.find(reading.sensorId);
        if (it != subscribers.end()) {
            for (auto& handler : it->second) {
                handler(reading);
            }
        }
    }
};

// Usage
IoTEventBus bus;

// Temperature monitor subscribes
bus.subscribe("temp_sensor_1", [](const SensorReading& r) {
    if (r.value > 30.0) {
        std::cout << "⚠️ High temperature: " << r.value << "°C\n";
    }
});

// Data logger subscribes
bus.subscribe("temp_sensor_1", [](const SensorReading& r) {
    logToDatabase(r);
// ...continues on next slide
```

📌 5. Event-Driven in Real-World C++ Systems (5/5)

[Code continues across slides]

```
// ...continued from previous slide
});

// Sensor publishes reading
SensorReading reading{"temp_sensor_1", 35.5, now()};
bus.publish(reading);
// Output:
// ⚠️ High temperature: 35.5°C
// [Database logged]
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 6. Event-Driven Benefits and Trade-offs (1/2)

Benefits:

Benefit	Explanation	Example
Loose Coupling	Producer doesn't know consumers	Button doesn't know who handles click
Scalability	Easy to add new consumers	Add logger without changing button
Flexibility	Change behavior at runtime	Enable/disable handlers dynamically
Asynchronous	Non-blocking operations	Publish event and continue
Multiple Handlers	Many consumers for one event	Click triggers UI + analytics + logging

Trade-offs:

Drawback	Explanation	Mitigation
Debugging	Hard to trace event flow	Event logging, tracing tools
Order Uncertainty	Don't know handler order	Use priority queue if order matters
Memory	Events queued consume memory	Bounded queues, backpressure
Complexity	More moving parts	Start simple, add complexity when needed
Performance	Event dispatching overhead	Profile, optimize hot paths

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 6. Event-Driven Benefits and Trade-offs (2/2)

When Event-Driven Makes Sense:

```
✅ Many-to-many communication (1 producer, N consumers)
✅ Loose coupling desired (components independent)
✅ Asynchronous operations (don't wait for response)
✅ Dynamic behavior (add/remove handlers at runtime)
✅ Real-time reactivity (respond to external events)

❌ Simple linear flow (just call function)
❌ Need return value immediately (synchronous call better)
❌ Performance critical (function call faster than event)
❌ Debugging critical (call stack clearer than events)
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 7. Event-Driven Design Patterns (1/4)

Pattern A: Observer Pattern (Classic)

```
class Subject {
    std::vector<Observer*> observers;

public:
    void attach(Observer* obs) {
        observers.push_back(obs);
    }

    void notify() {
        for (auto* obs : observers) {
            obs->update(this);
        }
    }

    void setState(int s) {
        state = s;
        notify();  // Notify on state change
    }

private:
    int state;
};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 7. Event-Driven Design Patterns (2/4)

Pattern B: Publish-Subscribe (Decoupled)

```
class PubSub {
    std::map<std::string, std::vector<std::function<void(const std::string&)>>> subscribers;

public:
    void subscribe(const std::string& topic, std::function<void(const std::string&)> handler) {
        subscribers[topic].push_back(handler);
    }

    void publish(const std::string& topic, const std::string& message) {
        for (auto& handler : subscribers[topic]) {
            handler(message);
        }
    }
};

// Usage
PubSub bus;
bus.subscribe("news", [](const std::string& msg) { std::cout << "News: " << msg << "\n"; });
bus.subscribe("news", [](const std::string& msg) { logNews(msg); });

bus.publish("news", "Breaking news!");
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 7. Event-Driven Design Patterns (3/4)

Pattern C: Event Sourcing (Store All Events)

```
class EventStore {
    std::vector<Event> events;

public:
    void append(const Event& event) {
        events.push_back(event);
        // Also publish to handlers
        notifyHandlers(event);
    }

    std::vector<Event> getEvents() const {
        return events;  // Replay all events
    }

    void replay() {
        for (const auto& event : events) {
            notifyHandlers(event);
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Theory

📌 7. Event-Driven Design Patterns (4/4)

Pattern D: Command Pattern (Event as Command)

```
class Command {
public:
    virtual void execute() = 0;
    virtual void undo() = 0;
};

class MoveCommand : public Command {
    int* position;
    int delta;

public:
    MoveCommand(int* pos, int d) : position(pos), delta(d) {}

    void execute() override {
        *position += delta;
    }

    void undo() override {
        *position -= delta;
    }
};

// Event bus for commands
std::queue<std::unique_ptr<Command>> commandQueue;

// Publish command
commandQueue.push(std::make_unique<MoveCommand>(&playerX, 10));

// Process commands
while (!commandQueue.empty()) {
    commandQueue.front()->execute();
    commandQueue.pop();
}
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 1. Event Handler Modifies Subscriber List (Iterator Invalidation) (1/4)

Problem: Handler unsubscribes while event is being dispatched, invalidating iterator.

```
// ❌ DANGEROUS: Iterator invalidation

class EventBus {
    std::vector<EventHandler> handlers;

public:
    void publish(const Event& event) {
        for (auto& handler : handlers) {  // ← Iterator
            handler(event);
            // What if handler calls unsubscribe()?
            // Iterator invalidated! Crash!
        }
    }

    void unsubscribe(EventHandler h) {
        handlers.erase(std::remove(handlers.begin(), handlers.end(), h), handlers.end());
        // ❌ Modifies vector while iterating!
    }
};

// Handler that unsubscribes itself
bus.subscribe([&bus](const Event& e) {
    std::cout << "Handle once\n";
    bus.unsubscribe(/* this handler */);  // ❌ Crashes!
});
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 1. Event Handler Modifies Subscriber List (Iterator Invalidation) (2/4)

Solution 1: Copy Handlers Before Iterating

```
class EventBus {
    std::vector<EventHandler> handlers;

public:
    void publish(const Event& event) {
        // Copy handlers (avoid iterator invalidation)
        auto handlersCopy = handlers;  // ✅ Snapshot

        for (auto& handler : handlersCopy) {
            handler(event);  // Safe - iterating copy
        }
    }

    void unsubscribe(EventHandler h) {
        handlers.erase(std::remove(handlers.begin(), handlers.end(), h), handlers.end());
        // OK - modifies original, not copy
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 1. Event Handler Modifies Subscriber List (Iterator Invalidation) (3/4)

Solution 2: Mark for Removal, Clean Later

```
class EventBus {
    std::vector<std::pair<EventHandler, bool>> handlers;  // handler, active
    bool dispatching = false;

public:
    void publish(const Event& event) {
        dispatching = true;

        for (auto& [handler, active] : handlers) {
            if (active) {
                handler(event);
            }
        }

        dispatching = false;
        cleanup();  // Remove marked handlers
    }

    void unsubscribe(EventHandler h) {
        if (dispatching) {
            // Mark for removal
            for (auto& [handler, active] : handlers) {
                if (handler == h) {
                    active = false;  // ✅ Just mark, don't remove
                }
            }
        } else {
            // Remove immediately
            handlers.erase(std::remove_if(handlers.begin(), handlers.end(),
                [&h](const auto& p) { return p.first == h; }), handlers.end());
        }
    }

private:
    void cleanup() {
        handlers.erase(std::remove_if(handlers.begin(), handlers.end(),
            [](const auto& p) { return !p.second; }), handlers.end());
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 1. Event Handler Modifies Subscriber List (Iterator Invalidation) (4/4)

Why This Matters: Modifying collections during iteration is a common bug in event systems. Always protect against it.

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 2. Event Loops (Infinite Recursion) (1/4)

Problem: Event handler publishes same event, causing infinite loop.

```
// ❌ INFINITE LOOP!

class DataProcessor {
    EventBus* bus;

public:
    DataProcessor(EventBus* b) : bus(b) {
        bus->subscribe(EventType::DataChanged, [this](const Event& e) {
            processData(e);
        });
    }

    void processData(const Event& e) {
        // Process data...

        // Publish event
        bus->publish(Event{EventType::DataChanged, "self"});  // ❌ Triggers self!
        // Infinite recursion!
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 2. Event Loops (Infinite Recursion) (2/4)

Solution 1: Track Recursion Depth

```
class EventBus {
    int recursionDepth = 0;
    const int MAX_DEPTH = 10;

public:
    void publish(const Event& event) {
        if (recursionDepth >= MAX_DEPTH) {
            std::cerr << "⚠️ Event loop detected! Depth: " << recursionDepth << "\n";
            return;  // Abort
        }

        ++recursionDepth;

        for (auto& handler : handlers[event.type]) {
            handler(event);
        }

        --recursionDepth;
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 2. Event Loops (Infinite Recursion) (3/4)

Solution 2: Defer Recursive Events

```
class EventBus {
    std::queue<Event> deferredEvents;
    bool dispatching = false;

public:
    void publish(const Event& event) {
        if (dispatching) {
            // Already dispatching - defer this event
            deferredEvents.push(event);
            return;
        }

        dispatching = true;
        dispatch(event);

        // Process deferred events
        while (!deferredEvents.empty()) {
            Event deferred = deferredEvents.front();
            deferredEvents.pop();
            dispatch(deferred);
        }

        dispatching = false;
    }

private:
    void dispatch(const Event& event) {
        for (auto& handler : handlers[event.type]) {
            handler(event);
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 2. Event Loops (Infinite Recursion) (4/4)

Solution 3: Prevent Self-Triggering

```
void processData(const Event& e) {
    // Check if event came from self
    if (e.source == "self") {
        return;  // Ignore self-triggered events
    }

    // Process...
    bus->publish(Event{EventType::DataChanged, "self"});  // Mark as self
}
```

Why This Matters: Event loops are hard to debug and can crash your program. Always guard against infinite recursion.

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 3. Event Ordering and Race Conditions (1/4)

Problem: Events processed out of order lead to inconsistent state.

```
// ❌ RACE CONDITION

class BankAccount {
    double balance = 1000.0;
    EventBus* bus;

public:
    void withdraw(double amount) {
        // Publish event (asynchronous)
        bus->publish(Event{EventType::Withdrawal, &amount});
    }

    void onWithdrawal(double amount) {
        // This might be called out of order!
        if (balance >= amount) {
            balance -= amount;
            std::cout << "Withdrew $" << amount << ", balance: $" << balance << "\n";
        } else {
            std::cout << "Insufficient funds!\n";
        }
    }
};

// Multiple withdrawals
account.withdraw(600);  // Event 1
account.withdraw(500);  // Event 2

// Events might process as: Event 2, then Event 1
// Result: Both succeed! (balance went negative)
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 3. Event Ordering and Race Conditions (2/4)

Solution 1: Synchronous Critical Operations

```
void withdraw(double amount) {
    // Critical operation - do synchronously
    if (balance >= amount) {
        balance -= amount;

        // Publish after operation completes
        bus->publish(Event{EventType::WithdrawalComplete, &amount});
    }
}
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 3. Event Ordering and Race Conditions (3/4)

Solution 2: Sequence Numbers

```
struct SequencedEvent {
    Event event;
    uint64_t sequence;

    bool operator<(const SequencedEvent& other) const {
        return sequence > other.sequence;  // Min heap
    }
};

class SequencedEventBus {
    std::priority_queue<SequencedEvent> queue;
    uint64_t nextSequence = 0;
    uint64_t processedSequence = 0;

public:
    void publish(const Event& event) {
        queue.push({event, nextSequence++});
    }

    void processEvents() {
        while (!queue.empty() && queue.top().sequence == processedSequence) {
            SequencedEvent se = queue.top();
            queue.pop();

            dispatch(se.event);
            processedSequence++;
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 3. Event Ordering and Race Conditions (4/4)

Solution 3: Lock for Critical State

```
class BankAccount {
    double balance = 1000.0;
    std::mutex balanceMutex;

    void onWithdrawal(double amount) {
        std::lock_guard lock(balanceMutex);  // ✅ Serialize access

        if (balance >= amount) {
            balance -= amount;
        }
    }
};
```

Why This Matters: Asynchronous events can arrive out of order. Use sequence numbers, locks, or make critical operations synchronous.

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 4. Memory Leaks from Dangling Handlers (1/3)

Problem: Object destroyed but handler still registered, causing use-after-free or memory leak.

```
// ❌ DANGLING POINTER

class Logger {
    EventBus* bus;

public:
    Logger(EventBus* b) : bus(b) {
        bus->subscribe(EventType::Log, [this](const Event& e) {
            this->log(e);  // ❌ 'this' captured!
        });
    }

    void log(const Event& e) {
        std::cout << "Log: " << e.data << "\n";
    }

    ~Logger() {
        // ❌ Forgot to unsubscribe!
    }
};

{
    Logger logger(&bus);
}  // logger destroyed

bus.publish(Event{EventType::Log, "message"});  // ❌ Calls destroyed logger!
// Use-after-free! Crash!
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 4. Memory Leaks from Dangling Handlers (2/3)

Solution 1: Unsubscribe in Destructor

```
class Logger {
    EventBus* bus;
    EventHandler myHandler;

public:
    Logger(EventBus* b) : bus(b) {
        myHandler = [this](const Event& e) {
            this->log(e);
        };

        bus->subscribe(EventType::Log, myHandler);
    }

    ~Logger() {
        bus->unsubscribe(EventType::Log, myHandler);  // ✅ Clean up
    }
};
```

Solution 2: Use weak_ptr

```
class Logger : public std::enable_shared_from_this<Logger> {
public:
    Logger(EventBus* b) {
        bus->subscribe(EventType::Log,
            [weak = std::weak_ptr<Logger>(shared_from_this())](const Event& e) {
                if (auto strong = weak.lock()) {  // ✅ Check if still alive
                    strong->log(e);
                } else {
                    // Object destroyed, skip handler
                }
            }
        );
    }
};

// Usage
auto logger = std::make_shared<Logger>(&bus);
// When logger destroyed, handler automatically becomes no-op
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 4. Memory Leaks from Dangling Handlers (3/3)

Solution 3: RAII Subscription Handle

```
class Subscription {
    EventBus* bus;
    EventType type;
    EventHandler handler;

public:
    Subscription(EventBus* b, EventType t, EventHandler h)
        : bus(b), type(t), handler(h) {
        bus->subscribe(type, handler);
    }

    ~Subscription() {
        bus->unsubscribe(type, handler);  // ✅ Auto-unsubscribe
    }

    // Prevent copying
    Subscription(const Subscription&) = delete;
    Subscription& operator=(const Subscription&) = delete;
};

class Logger {
    Subscription sub;

public:
    Logger(EventBus* bus)
        : sub(bus, EventType::Log, [this](const Event& e) { log(e); }) {}

    // ~Logger() automatically unsubscribes via sub's destructor
};
```

Why This Matters: Forgetting to unsubscribe causes use-after-free bugs. Use RAII or weak_ptr to manage handler lifetime.

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 5. Event Storms (Too Many Events) (1/4)

Problem: Burst of events overwhelms system, causing memory exhaustion or slowdown.

```
// ❌ EVENT STORM

// Sensor publishes 1000 events/second
for (int i = 0; i < 1000; ++i) {
    bus.publish(Event{EventType::SensorData, data});
}

// Event queue grows unbounded
// Memory: 1MB/event * 1000 events = 1GB memory!
// Handlers can't keep up
```

Solution 1: Bounded Queue with Backpressure

```
class BoundedEventBus {
    std::queue<Event> eventQueue;
    const size_t MAX_QUEUE_SIZE = 1000;
    std::mutex queueMutex;

public:
    bool publish(const Event& event) {
        std::lock_guard lock(queueMutex);

        if (eventQueue.size() >= MAX_QUEUE_SIZE) {
            std::cerr << "⚠️ Event queue full! Dropping event.\n";
            return false;  // ✅ Reject event (backpressure)
        }

        eventQueue.push(event);
        return true;
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 5. Event Storms (Too Many Events) (2/4)

Solution 2: Event Coalescing (Merge Similar Events)

```
class CoalescingEventBus {
    std::map<EventType, Event> latestEvents;  // Only keep latest

public:
    void publish(const Event& event) {
        // Replace old event with new one
        latestEvents[event.type] = event;  // ✅ Only 1 event per type
    }

    void processEvents() {
        for (auto& [type, event] : latestEvents) {
            dispatch(event);
        }
        latestEvents.clear();
    }
};

// Result: 1000 sensor events → 1 event (latest)
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 5. Event Storms (Too Many Events) (3/4)

Solution 3: Rate Limiting

```
class RateLimitedEventBus {
    std::map<EventType, uint64_t> lastPublishTime;
    const uint64_t MIN_INTERVAL_MS = 100;  // Max 10 events/sec per type

public:
    bool publish(const Event& event) {
        uint64_t now = getCurrentTimeMs();
        uint64_t lastTime = lastPublishTime[event.type];

        if (now - lastTime < MIN_INTERVAL_MS) {
            // Too soon, drop event
            return false;  // ✅ Rate limited
        }

        lastPublishTime[event.type] = now;
        actuallyPublish(event);
        return true;
    }
};
```

Solution 4: Sampling (Process Every Nth Event)

```
class SamplingEventBus {
    std::map<EventType, int> eventCounts;
    const int SAMPLE_RATE = 10;  // Process every 10th event

public:
    void publish(const Event& event) {
        int& count = eventCounts[event.type];
        count++;

        if (count % SAMPLE_RATE == 0) {
            actuallyPublish(event);  // ✅ Only process 10% of events
        }
    }
};
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 5. Event Storms (Too Many Events) (4/4)

Why This Matters: Unbounded event queues can exhaust memory. Use bounded queues, coalescing, rate limiting, or sampling.

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 6. Exception Safety in Event Handlers (1/3)

Problem: Handler throws exception, stopping event propagation to other handlers.

```
// ❌ ONE BAD HANDLER BREAKS EVERYTHING

void publish(const Event& event) {
    for (auto& handler : handlers) {
        handler(event);  // ❌ If throws, remaining handlers not called!
    }
}

// Handler 1: Works
bus.subscribe([](const Event& e) {
    std::cout << "Handler 1\n";
});

// Handler 2: Throws
bus.subscribe([](const Event& e) {
    throw std::runtime_error("Oops!");  // ❌ Throws!
});

// Handler 3: Never called
bus.subscribe([](const Event& e) {
    std::cout << "Handler 3\n";  // ❌ Never reaches here
});

bus.publish(event);
// Output: Handler 1
// Exception! Handler 3 never runs
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 6. Exception Safety in Event Handlers (2/3)

Solution 1: Catch and Log Exceptions

```
void publish(const Event& event) {
    for (auto& handler : handlers) {
        try {
            handler(event);  // ✅ Try each handler
        } catch (const std::exception& e) {
            std::cerr << "Handler exception: " << e.what() << "\n";
            // Continue to next handler
        } catch (...) {
            std::cerr << "Handler exception: unknown\n";
        }
    }
    // All handlers called, even if some throw
}
```

Solution 2: Collect Exceptions, Rethrow Later

```
void publish(const Event& event) {
    std::vector<std::exception_ptr> exceptions;

    for (auto& handler : handlers) {
        try {
            handler(event);
        } catch (...) {
            exceptions.push_back(std::current_exception());
        }
    }

    // All handlers called

    if (!exceptions.empty()) {
        // Rethrow first exception
        std::rethrow_exception(exceptions[0]);
    }
}
```

Event-Driven Architecture (Asynchronous Systems)

Edge Cases

📌 6. Exception Safety in Event Handlers (3/3)

Solution 3: Error Event

```
void publish(const Event& event) {
    for (auto& handler : handlers) {
        try {
            handler(event);
        } catch (const std::exception& e) {
            // Publish error event
            Event errorEvent{EventType::HandlerError, e.what()};
            publishError(errorEvent);  // Separate error handling
        }
    }
}
```

Why This Matters: One misbehaving handler shouldn't break all others. Always catch exceptions in event dispatch.

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 1. Easy - Simple Event System (Synchronous) (1/3)

```
#include <iostream>
#include <vector>
#include <functional>
#include <map>

// Simple event type
enum class EventType {
    ButtonClick,
    KeyPress,
    MouseMove
};

// Event data
struct Event {
    EventType type;
    std::string data;
};

// Event handler type
using EventHandler = std::function<void(const Event&)>;

// Simple event bus (synchronous)
class EventBus {
    std::map<EventType, std::vector<EventHandler>> subscribers;

public:
    // Subscribe to events
    void subscribe(EventType type, EventHandler handler) {
        subscribers[type].push_back(handler);
    }

    // Publish event (calls all handlers immediately)
    void publish(const Event& event) {
        auto it = subscribers.find(event.type);
        if (it != subscribers.end()) {
            for (auto& handler : it->second) {
                handler(event);  // Call handler
            }
        }
    }
};
// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 1. Easy - Simple Event System (Synchronous) (2/3)

```
// ...continued from previous slide

int main() {
    std::cout << "Simple Event System Example\n\n";

    EventBus bus;

    // Subscribe to button clicks
    bus.subscribe(EventType::ButtonClick, [](const Event& e) {
        std::cout << "[Handler 1] Button clicked: " << e.data << "\n";
    });

    bus.subscribe(EventType::ButtonClick, [](const Event& e) {
        std::cout << "[Handler 2] Processing click: " << e.data << "\n";
    });

    // Subscribe to key presses
    bus.subscribe(EventType::KeyPress, [](const Event& e) {
        std::cout << "[Handler 3] Key pressed: " << e.data << "\n";
    });

    // Publish events
    std::cout << "Publishing ButtonClick event:\n";
    bus.publish(Event{EventType::ButtonClick, "LoginButton"});

    std::cout << "\nPublishing KeyPress event:\n";
    bus.publish(Event{EventType::KeyPress, "Enter"});

    return 0;
}

/*
Output:
Simple Event System Example

Publishing ButtonClick event:
[Handler 1] Button clicked: LoginButton
[Handler 2] Processing click: LoginButton

Publishing KeyPress event:
[Handler 3] Key pressed: Enter
// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 1. Easy - Simple Event System (Synchronous) (3/3)

[Code continues across slides]

```
// ...continued from previous slide

Key Points:
- Synchronous: Handlers called immediately
- Multiple handlers: Both handlers run for ButtonClick
- Simple: Easy to understand and debug
*/
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 2. Mid - Queued Event System (Asynchronous) (1/3)

```
#include <iostream>
#include <queue>
#include <functional>
#include <map>

enum class EventType { SensorData, UserInput, TimerTick };

struct Event {
    EventType type;
    int value;
    uint64_t timestamp;
};

using EventHandler = std::function<void(const Event&)>;

// Queued event bus (asynchronous)
class QueuedEventBus {
    std::queue<Event> eventQueue;
    std::map<EventType, std::vector<EventHandler>> subscribers;

public:
    void subscribe(EventType type, EventHandler handler) {
        subscribers[type].push_back(handler);
    }

    // Publish adds to queue (returns immediately)
    void publish(const Event& event) {
        eventQueue.push(event);
        std::cout << "  [BUS] Event queued (queue size: " << eventQueue.size() << ")\n";
    }

    // Process queued events (called explicitly)
    void processEvents() {
        std::cout << "\n[BUS] Processing " << eventQueue.size() << " events...\n\n";

        while (!eventQueue.empty()) {
            Event e = eventQueue.front();
            eventQueue.pop();

            // Dispatch to handlers
            auto it = subscribers.find(e.type);
// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 2. Mid - Queued Event System (Asynchronous) (2/3)

```
// ...continued from previous slide
            if (it != subscribers.end()) {
                for (auto& handler : it->second) {
                    handler(e);
                }
            }
        }
    }
};

int main() {
    std::cout << "Queued Event System Example\n";
    std::cout << "==========================\n\n";

    QueuedEventBus bus;

    // Subscribe handlers
    bus.subscribe(EventType::SensorData, [](const Event& e) {
        std::cout << "  [HANDLER] Sensor reading: " << e.value << "\n";
    });

    bus.subscribe(EventType::UserInput, [](const Event& e) {
        std::cout << "  [HANDLER] User input: " << e.value << "\n";
    });

    // Simulate game loop
    for (int frame = 0; frame < 3; ++frame) {
        std::cout << "═══ Frame " << frame << " ═══\n";

        // Collect events (non-blocking)
        std::cout << "Collecting events...\n";
        bus.publish(Event{EventType::SensorData, frame * 10, 0});
        bus.publish(Event{EventType::UserInput, frame, 0});

        // Do other work
        std::cout << "Doing game logic...\n";
        std::cout << "Rendering...\n";

        // Process all queued events
        bus.processEvents();

// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 2. Mid - Queued Event System (Asynchronous) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        std::cout << "\n";
    }

    return 0;
}

/*
Output:
Queued Event System Example
===========================

=== Frame 0 ===
Collecting events...
  [BUS] Event queued (queue size: 1)
  [BUS] Event queued (queue size: 2)
Doing game logic...
Rendering...

[BUS] Processing 2 events...

  [HANDLER] Sensor reading: 0
  [HANDLER] User input: 0

=== Frame 1 ===
[Similar...]

Key Points:
- Asynchronous: publish() returns immediately
- Queued: Events processed in batch
- Control: You decide when to process events
- Game loops: Common pattern in games
*/
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 3. Mid - Priority Event System (1/3)

Continuing in next section due to length...

```
#include <iostream>
#include <queue>
#include <functional>
#include <map>

enum class EventType { Emergency, High, Normal, Low };

struct PriorityEvent {
    Event event;
    int priority;  // Higher = more urgent

    bool operator<(const PriorityEvent& other) const {
        return priority < other.priority;  // Max heap (highest first)
    }
};

struct Event {
    EventType type;
    std::string message;
};

using EventHandler = std::function<void(const Event&)>;

class PriorityEventBus {
    std::priority_queue<PriorityEvent> eventQueue;
    std::map<EventType, std::vector<EventHandler>> subscribers;

public:
    void subscribe(EventType type, EventHandler handler) {
        subscribers[type].push_back(handler);
    }

    void publish(const Event& event, int priority) {
        eventQueue.push({event, priority});
        std::cout << "  [QUEUE] Added " << event.message
                  << " (priority: " << priority << ")\n";
    }

    void processEvents() {
// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 3. Mid - Priority Event System (2/3)

```
// ...continued from previous slide
        std::cout << "\n[PROCESSING] Events in priority order:\n\n";

        while (!eventQueue.empty()) {
            PriorityEvent pe = eventQueue.top();
            eventQueue.pop();

            std::cout << "  → Processing (priority " << pe.priority << "): "
                      << pe.event.message << "\n";

            auto it = subscribers.find(pe.event.type);
            if (it != subscribers.end()) {
                for (auto& handler : it->second) {
                    handler(pe.event);
                }
            }
        }
    }
};

int main() {
    std::cout << "Priority Event System Example\n";
    std::cout << "=============================\n\n";

    PriorityEventBus bus;

    bus.subscribe(EventType::Emergency, [](const Event& e) {
        std::cout << "    🚨 EMERGENCY HANDLER: " << e.message << "\n";
    });

    bus.subscribe(EventType::Normal, [](const Event& e) {
        std::cout << "    ℹ️  Normal handler: " << e.message << "\n";
    });

    // Publish events in random order
    std::cout << "Publishing events (out of order):\n";
    bus.publish({EventType::Normal, "Log message"}, 1);
    bus.publish({EventType::Emergency, "Collision detected!"}, 10);  // High priority
    bus.publish({EventType::Normal, "Status update"}, 1);
    bus.publish({EventType::Emergency, "System critical!"}, 10);

// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 3. Mid - Priority Event System (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    // Process in priority order
    bus.processEvents();

    return 0;
}

/*
Output:
Priority Event System Example
============================

Publishing events (out of order):
  [QUEUE] Added Log message (priority: 1)
  [QUEUE] Added Collision detected! (priority: 10)
  [QUEUE] Added Status update (priority: 1)
  [QUEUE] Added System critical! (priority: 10)

[PROCESSING] Events in priority order:

  → Processing (priority 10): Collision detected!
    🚨 EMERGENCY HANDLER: Collision detected!
  → Processing (priority 10): System critical!
    🚨 EMERGENCY HANDLER: System critical!
  → Processing (priority 1): Log message
    ℹ️  Normal handler: Log message
  → Processing (priority 1): Status update
    ℹ️  Normal handler: Status update

Key Points:
- Priority-based: High-priority events processed first
- Critical systems: Emergencies handled before normal events
- Real-time: Important for safety-critical systems
*/
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 4. Advanced - Thread-Safe Event Bus (1/3)

```
#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <map>
#include <functional>
#include <chrono>

enum class EventType { DataReady, ProcessComplete };

struct Event {
    EventType type;
    int data;
    int threadId;
};

using EventHandler = std::function<void(const Event&)>;

// Thread-safe event bus
class ThreadSafeEventBus {
    std::map<EventType, std::vector<EventHandler>> subscribers;
    std::mutex subscribersMutex;

public:
    void subscribe(EventType type, EventHandler handler) {
        std::lock_guard lock(subscribersMutex);
        subscribers[type].push_back(handler);
    }

    void publish(const Event& event) {
        // Copy handlers under lock (minimize lock time)
        std::vector<EventHandler> handlersCopy;
        {
            std::lock_guard lock(subscribersMutex);
            auto it = subscribers.find(event.type);
            if (it != subscribers.end()) {
                handlersCopy = it->second;
            }
        }  // Release lock

        // Call handlers without holding lock
// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 4. Advanced - Thread-Safe Event Bus (2/3)

```
// ...continued from previous slide
        for (auto& handler : handlersCopy) {
            handler(event);
        }
    }
};

int main() {
    std::cout << "Thread-Safe Event Bus Example\n";
    std::cout << "==============================\n\n";

    ThreadSafeEventBus bus;

    // Subscribe handlers
    bus.subscribe(EventType::DataReady, [](const Event& e) {
        std::cout << "  [Thread " << std::this_thread::get_id()
                  << "] Data received: " << e.data
                  << " from thread " << e.threadId << "\n";
    });

    // Launch multiple producer threads
    std::vector<std::thread> threads;

    for (int i = 0; i < 5; ++i) {
        threads.emplace_back([&bus, i]() {
            // Simulate work
            std::this_thread::sleep_for(std::chrono::milliseconds(100 * i));

            // Publish event from this thread
            Event e{EventType::DataReady, i * 10, i};
            bus.publish(e);
        });
    }

    // Wait for all threads
    for (auto& t : threads) {
        t.join();
    }

    std::cout << "\n✅ All threads completed safely!\n";

// ...continues on next slide
```

Event-Driven Architecture (Asynchronous Systems)

Code Examples

📌 4. Advanced - Thread-Safe Event Bus (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    return 0;
}

/*
Output:
Thread-Safe Event Bus Example
==============================

  [Thread 140234] Data received: 0 from thread 0
  [Thread 140235] Data received: 10 from thread 1
  [Thread 140236] Data received: 20 from thread 2
  [Thread 140237] Data received: 30 from thread 3
  [Thread 140238] Data received: 40 from thread 4

✅ All threads completed safely!

Key Points:
- Thread-safe: Multiple threads can publish concurrently
- Lock efficiency: Hold lock only while copying handlers
- No data races: Mutex protects shared data
- Production use: Required for multi-threaded systems
*/
```

What You Learned

Key Takeaways

- ✓ What is Event-Driven Architecture? (Simple Explanation)
- ✓ Core Components of Event-Driven Systems
- ✓ Event-Driven Patterns
- ✓ Synchronous vs Asynchronous Event Handling

→ Swipe to learn →